Architecture Validation

How Hype's planned architecture compares to Yelp, TripAdvisor, Google Maps, Airbnb, Fever, Resident Advisor, DICE, and Wanderlog. Based on public engineering blogs, API docs, and conference talks.

1. How the Industry Does AI

APP	AI FEATURES	AI SERVER-SIDE?	MODEL STRATEGY	CACHING	BACKEND
Yelp	AI Assistant (conversational search), review summaries, photo captions	Yes — LLM Gateway	Tiered: GPT-4.1-nano for routing/safety, GPT-4o for generation	Heavy — pre-computed summaries	Kubernetes (PaaSTA)
TripAdvisor	AI Trip Builder, review summaries, itinerary generation	Yes — RAG pipeline	RAG: LLM translates intent → DB query → formats results	Trip plans stored per user	Microservices
Google Maps	Place summaries, area summaries, route suggestions	Yes — Gemini API	Gemini models, pre-computed via Places API	Heavy — summaries are API fields	Google Cloud
Airbnb	AI search, photo auto-tagging, listing descriptions, review summaries	Yes — SSE streaming	Multi-model, 1M events/sec behavioral pipeline	Per-session, streaming	Kubernetes + Kafka + Flink
Fever	Personalized recommendations (behavioral), secret plans	Yes	ML models on 300M+ monthly interactions, not LLM-heavy	Behavioral scores pre-computed	Microservices
Resident Advisor	Music taste → event matching (via Spotify connect)	Yes	Traditional ML, Spotify signal integration	Preference profiles	Microservices
DICE	Personalized event feed, dynamic pricing signals	Yes	Traditional ML recommendations	Feed pre-ranked	Elixir + Helm/K8s
Wanderlog	AI itinerary generation, place suggestions	Yes	LLM + structured place database	Generated itineraries stored	Serverless + DB
Hype OURS	AI planner, city pulse, smart search, surprise me, live translation, Instagram parsing	Yes — Edge Functions	Multi-provider: GPT-4.1 mini/nano, Gemini Flash, Claude Haiku	Pulse cached 3h, plans stored per user	Supabase Edge Functions + Node backend

2. Architecture Comparison

INDUSTRY STANDARD (YELP-SCALE)

Microservices + LLM Gateway

```
Client App
  → API Gateway (auth, rate limit)
```

HYPE (STARTUP-SCALE)

Supabase + Edge Functions + Node Backend

```
Expo App
  → Supabase Client (auth, direct reads)
```

```
→ Search Service (Elasticsearch)
→ Recommendation Service (ML models)
→ LLM Gateway
  → Intent Classifier (cheap model)
  → Safety Filter (cheap model)
  → Generator (expensive model)
  → Response Cache (Redis)
→ Data Service (PostgreSQL)
→ Event Bus (Kafka)
→ Analytics Pipeline (Flink)
```

Dozens of engineers. Millions in infra. Years to build.

```
→ PostgreSQL (venues, events, profiles)
→ Edge Functions (AI proxy layer)
  → generate-plan → GPT-4.1 mini
  → generate-pulse → Gemini Flash-Lite
  → smart-search → GPT-4.1 nano
  → surprise-me → GPT-4.1 mini
  → translate-scene → Gemini Flash
  → parse-instagram → Claude Haiku
→ Node Backend (ingestion, scraping)
  → Scheduled scrapes → raw_events
  → Instagram pipeline → event extraction
  → Google Maps photos → venue images
```

Same AI patterns. 1/100th the infrastructure.

3. Patterns to Steal

STEAL

LLM as Translator

From: Yelp Assistant

The LLM doesn't know your data — it translates user intent into structured queries against your real database. "Good coffee near Baščaršija" → `{category: 'cafe', neighborhood: 'Baščaršija', sort: 'rating'}`. The DB does the real work.

STEAL

Tiered Model Routing

From: Yelp, Airbnb

Use the cheapest model that works for each task. GPT-4.1 nano ($0.10/1M) for intent classification and search parsing. GPT-4.1 mini ($0.40/1M) only for creative generation (plans, surprise me). Never send a $3 query when a $0.10 one works.

STEAL

Pre-computed Summaries

From: Google Maps Places API

Google doesn't generate place summaries on every request — they pre-compute and store them as API fields. Hype should do the same: batch-generate venue descriptions, cache city pulse blurbs, store plan results. Real-time AI only where it matters.

ADAPT

SSE Streaming for Plans

From: Airbnb AI Search

Airbnb streams AI search results via Server-Sent Events so users see results appearing word-by-word. Apply this to the Tonight Planner — stream the plan as it generates instead of showing a loading spinner for 3 seconds. Much more theatrical.

ADAPT

Behavioral Signals Feed

From: Fever (300M interactions/mo)

Fever's moat is behavioral data, not algorithms. Hype should start collecting signals from day one: saves, search queries, mood selections, time-of-day usage, plan likes. Even if we don't act on them yet, the data compounds.

ADAPT

Music Taste → Event Match

From: Resident Advisor

RA connects Spotify to match music taste with events. Hype could do this later for club/concert recommendations. Not for the presentation, but worth designing the mood system to accommodate music signals in the future.

4. The Verdict

Hype's Architecture is Sound

Every successful app in this space runs AI server-side. Hype's Edge Function pattern matches this exactly. API keys never touch the client. AI calls are proxied through a controlled layer.

Every successful app uses tiered models. Hype's multi-provider strategy (GPT-4.1 nano for cheap tasks, mini for creative, Gemini for vision, Claude for prose) matches the Yelp pattern at a fraction of the cost.

RAG beats fine-tuning. Hype already has the right instinct: feed real venue/event data to the LLM as context, don't try to teach the model about Sarajevo. The DB is the source of truth.

Caching is where the savings are. City pulse cached for 3 hours, venue descriptions batch-generated, plan results stored — this matches Google's pre-computed summaries pattern.

Supabase Edge Functions are appropriate for startup scale. No successful app at this stage runs Kubernetes. They all started simpler. Edge Functions give us the same AI proxy pattern without the infrastructure overhead. When Hype needs to scale beyond what Edge Functions handle, the migration path is clear: move AI orchestration to a dedicated service (Cloud Run / Railway).

One Thing to Watch

Edge Function cold starts and timeouts. Supabase Edge Functions (Deno) have ~25s execution limits on the free tier. The Chronicles project already handles this with `AbortController` timeouts. For the Tonight Planner (which needs to query Supabase for venues, build a prompt, call GPT, and return), we should design the prompt to be tight and the response to stream via SSE if possible. If a plan generation takes >10s, the user should see progress, not a spinner.